```
#include <iostream>
#include <iomanip>
#include <array>
#include <string>
using namespace std;

constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

enum class Zone {
  Inner = 0,
  Middle = 1,
  Outer = 2,
  Airport = 3
};

// Base fares: Rs. 10, Rs. 20, Rs. 30 and Rs. 60
constexpr array<long long, 4> BASE_FARE_PAISE = {
  1000, 2000, 3000, 6000
};

// Monthly pass allows unlimited journeys in two selected zones
class MonthlyPass {
private:
  Zone zoneOne;
  Zone zoneTwo;
  bool active;
```

```
public:
    MonthlyPass(Zone first, Zone second, bool isActive)
        : zoneOne(first), zoneTwo(second), active(isActive) {}

    bool covers(Zone zone) const {
        return active &&
               (zone == zoneOne || zone == zoneTwo);
    }
};

class FareCalculator {
public:
    static long long calculateFare(
        Zone zone, bool peak, bool weekend,
        const MonthlyPass& pass) {

        int index = static_cast<int>(zone);

        if (index < 0 ||
            index >= static_cast<int>(BASE_FARE_PAISE.size())) {
            return -1;
        }

        // Monthly pass: no fare for covered zones
        if (pass.covers(zone)) {
            return 0;
        }
```

```
        long long fare = BASE_FARE_PAISE[index];

        // Weekend discount and peak surcharge never combine.
        if (weekend) {
            fare = fare * (100 - WEEKEND_DISCOUNT_PERCENT) / 100;
        } else if (peak) {
            fare = fare * (100 + PEAK_SURCHARGE_PERCENT) / 100;
        }

        return fare;
    }
};

class SmartCard {
private:
    long long balancePaise;

public:
    explicit SmartCard(long long initialBalance)
        : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}

    long long getBalance() const {
        return balancePaise;
    }

    bool canTravel(long long fare) const {
        return fare >= 0 &&
```

```
            balancePaise >= fare &&
            balancePaise - fare >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare)) {
            return false;
        }

        balancePaise -= fare;
        return true;
    }
};

void printMoney(long long paise) {
    cout << "Rs. " << paise / 100 << "."
         << setw(2) << setfill('0') << paise % 100
         << setfill(' ');
}

int main() {
    cout << "NAMMA METRO - REVISED FARE SYSTEM\n\n";

    // Pass covers Inner and Middle zones
    MonthlyPass pass(Zone::Inner, Zone::Middle, true);

    SmartCard card(10000); // Rs. 100
```

```
struct Journey {
   string name;
   Zone zone;
   bool peak;
   bool weekend;
};

Journey journeys[] = {
   {"Inner zone weekday", Zone::Inner, false, false},
   {"Outer zone peak hour", Zone::Outer, true, false},
   {"Airport zone weekend", Zone::Airport, true, true},
   {"Middle zone monthly pass", Zone::Middle, true, false}
};

for (const Journey& journey : journeys) {
   long long fare = FareCalculator::calculateFare(
      journey.zone, journey.peak, journey.weekend, pass
   );

   cout << journey.name << " | Fare: ";

   if (fare < 0) {
      cout << "Invalid zone\n";
      continue;
   }

   printMoney(fare);
```

```
        if (card.deduct(fare)) {
            cout << " | Accepted | Balance: ";
            printMoney(card.getBalance());
            cout << '\n';
        } else {
            cout << " | Denied (minimum Rs. 50 must remain)"
                 << " | Balance: ";
            printMoney(card.getBalance());
            cout << '\n';
        }
    }

    cout << "\nFinal balance: ";
    printMoney(card.getBalance());
    cout << '\n';

    return 0;
}
```